Error handling

Язык программирования Rust. Лекция 5.

Where do errors come from?

Под ошибкой подразумеваем отклонение от *happy path*:

- Получение данных на вход в некорректном формате (например, от пользователя)
- Таймаут сетевого запроса
- Отсутствует файл на диске

Не считаем ошибками, которые нужно обрабатывать:

- Баги в коде (нарушенные инварианты, выход за границы массива) — их нужно исправлять, а не обрабатывать
- Закончилась память (по умолчанию Rust в этом случае аварийно завершает процесс)
- UB — это не ошибка, а нарушение контракта; поведение программы

Reasons to handle errors

- Такие ситуации точно возникнут, их нельзя отнести к невероятным.
- Залогировать/сообщить пользователю об ошибке, чтобы ее можно было исправить.
- Вовремя остановить исполнение программы, чтобы не сделать хуже.

Что хотим делать при возникновении ошибки?

- Сообщать пользователю/клиенту об ошибке.
- Изменять поведение программы, зачастую - ранний выход.
- Передавать ошибку в то место программы, где есть контекст для правильной обработки.

Different approaches to handle errors. Return codes

```
// POSIX: ssize_t из <sys/types.h>
ssize_t find(const int arr[], size_t len, int val) {
    for (size_t i = 0; i < len; i++) {
        if (arr[i] == val) return (ssize_t)i;
    }
    return -1; // not found
}
```

Return codes

```
type error interface {
    Error() string
}

func Open(name string) (file *File, err error)

f, err := os.Open("filename.ext")
if err != nil {
    log.Fatal(err)
}
// do something with the open *File f
```

Return codes: pros and cons

Pros:

- Производительность
- Не требуется специальной поддержки на уровне языка

Cons:

- Нет единообразия при обработке ошибок (в C: -1, NULL, `errno`, отрицательные коды...)
- Ошибка не является частью интерфейса - нужно тщательно читать документацию к функциям
- Иногда код ошибки нужно передавать рекурсивно по всему стеку вызовов
- Можно забыть обработать/передать выше ошибку

Exceptions

```
#include <cstdio>

struct A {
    A() { printf("A::A()\n"); }
    ~A() { printf("A::~A()\n"); }
};

void foo() { throw 42; }

void bar() {
    auto a = A{};
    foo();
    printf("all is well!");
}

int main() {
    try {
        bar();
    } catch (const int ex) {
        printf("exception");
    }
}
```

Exceptions: pros and cons

Pros:

- Нельзя забыть обработать ошибку (необработанное исключение завершит программу)
- Не нужно пробрасывать ошибку явно по стеку вызовов
- Хорошая читаемость кода за счет отделения happy path от логики обработки ошибок
- При наличии деструкторов получаем автоматическое освобождение ресурсов
- Нет накладных расходов на happy path (в реализациях с таблицами раскрутки, например в Itanium ABI)

Cons:

Rust error handling

- Unrecoverable errors - критические ошибки, после которых программа не может продолжать выполнение (ex. попытка достать out-of-bounds элемент из `Vec`).
- Recoverable errors - некритические ошибки (ex. file not found), при которых программа может работать дальше.

В Rust нет исключений: первые обрабатывает `panic!`, вторые — типы `Option<T>` и `Result<T, E>`.

Unrecoverable errors: `panic!`

```
fn main() {
    let mut argv = std::env::args();
    let arg: String = argv.nth(1).unwrap();
    let n: i32 = arg.parse().unwrap();
    println!("{}", 2 * n);
}
```

```
cargo run

thread 'main' panicked at src/main.rs:3:35:
called `Option::unwrap()` on a `None` value
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

`panic!` (1)

При возникновении паники:

- печатается сообщение об ошибке и backtrace (в дефолтной реализации panic hook, которую можно переопределить)
- раскручивается стек с вызовом деструкторов (`Drop`)
- завершается **поток**, в котором произошла паника. Если это главный поток — завершается процесс (код возврата 101). Если нет — остальные потоки продолжают работать, а `JoinHandle::join()` вернет `Err` с содержимым паники

`panic!` (2)

Поведение при панике можно поменять:

- panic hook
- поймать через `std::panic::catch_unwind`
- abort - завершение программы без раскрутки стека:

```
[profile.release]
panic = 'abort'
```

`panic!` backtrace

```
thread 'main' panicked at src/main.rs:3:35:
called `Option::unwrap()` on a `None` value
stack backtrace:
   0: __rustc::rust_begin_unwind
             at /rustc/1159e78c4747b02ef996e55082b704c09b970588/library/std/src/panicking.rs:697:5
   1: core::panicking::panic_fmt
             at /rustc/1159e78c4747b02ef996e55082b704c09b970588/library/core/src/panicking.rs:75:14
   2: core::panicking::panic
             at /rustc/1159e78c4747b02ef996e55082b704c09b970588/library/core/src/panicking.rs:145:5
   3: core::option::unwrap_failed
             at /rustc/1159e78c4747b02ef996e55082b704c09b970588/library/core/src/option.rs:2130:5
   4: core::option::Option<T>::unwrap
             at /Users/f.selishchev/.rustup/toolchains/stable-aarch64-apple-darwin/lib/rustlib/src/rust/library/core/src/option.rs:1009:21
   5: testing::main
             at ./src/main.rs:3:35
   6: core::ops::function::FnOnce::call_once
             at /Users/f.selishchev/.rustup/toolchains/stable-aarch64-apple-darwin/lib/rustlib/src/rust/library/core/src/ops/function.rs:253:5
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
```

Override `panic!` hook

`std` хук печатает сообщение об ошибке и backtrace.

Это поведение можно переопределить:

```
use std::panic;

panic::set_hook(Box::new(|_| {
    println!("Custom panic hook");
}));

panic!("Normal panic");
```

```
cargo run

Custom panic hook
```

Unset `panic!` hook

```rust
use std::panic;

panic::set_hook(Box::new(|_| {
    println!("Custom panic hook");
}));

let _ = panic::take_hook();

panic!("Normal panic");
```

```
cargo run

thread 'main' panicked at src/main.rs:10:5:
Normal panic
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

`std::panic::catch_unwind`

Ловит не все паники - только те, которые приводят к раскрутке стека (не сработает при `panic = 'abort'`, при панике во время раскрутки, при панике через `extern "C"`).

```
use std::panic;

// the closure should be UnwindSafe
// (otherwise wrap it into `AssertUnwindSafe`)
let result = panic::catch_unwind(|| {
    println!("hello!");
});
assert!(result.is_ok());

let result = panic::catch_unwind(|| {
    panic!("oh no!");
});
assert!(result.is_err());
```

RFC 1236: stabilize catch_panic

Recoverable errors

`Option<T>::unwrap()` -> `panic!`

```
enum Option<T> {
    None,
    Some(T),
}

impl<T> Option<T> {
    fn unwrap(self) -> T {
        match self {
            Some(val) => val,
            None => panic!("called `Option::unwrap()` on a `None` value"),
        }
    }
}
```

`unwrap()` not panic (1)

```
impl<T> Option<T> {
    fn unwrap_or(self, default: T) -> T {
        match self {
            Some(x) => x,
            None => default,
        }
    }

    fn unwrap_or_default(self) -> T
    where
        T: Default,
    {
        match self {
            Some(x) => x,
            None => T::default(),
        }
    }
}
```

`unwrap()` not panic (2)

```
impl<T> Option<T> {
    fn unwrap_or_else<F>(self, f: F) -> T
    where
        F: FnOnce() -> T,
    {
        match self {
            Some(x) => x,
            None => f(),
        }
    }
}
```

Combining `Option`s

```
fn find(haystack: &str, needle: char) -> Option<usize> {
    for (offset, c) in haystack.char_indices() {
        if c == needle {
            return Some(offset);
        }
    }
    None
}
fn extension(file_name: &str) -> Option<&str> {
    match find(file_name, '.') {
        None => None,
        Some(pos) => Some(&file_name[pos + 1..]),
    }
}
```

Combining `Option`s

```
impl<T> Option<T> {
    fn map<U, F>(self, f: F) -> Option<U>
    where
        F: FnOnce(T) -> U,
    {
        match self {
            Some(x) => Some(f(x)),
            None => None,
        }
    }
}

fn extension(file_name: &str) -> Option<&str> {
    find(file_name, '.').map(|i| &file_name[i + 1..])
}
```

Combining `Option`s

```
impl<T> Option<T> {
    fn and_then<U, F>(self, f: F) -> Option<U>
    where
        F: FnOnce(T) -> Option<U>,
    {
        match self {
            Some(x) => f(x),
            None => None,
        }
    }
}

fn file_name(file_path: &str) -> Option<&str> {
    todo!()
}

fn extension_from_file_path(file_path: &str) -> Option<&str> {
    file_name(file_path).and_then(extension)
}
```

`Result`

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

- `Result<T, E>` - обобщение `Option<T>`: вместе с отсутствием результата сообщает **причину** (`E`)
- `Option<T>` по смыслу эквивалентен `Result<T, ()>`:
  - `Some(x)` <-> `Ok(x)`
  - `None` <-> `Err(())`
- Это не `type`-алиас: в std `Option` и `Result` — два независимых enum'а, преобразуются друг в друга методами (следующий слайд)

`Result` <-> `Option`

```
impl<T, E> Result<T, E> {
    fn ok(self) -> Option<T> {
        match self {
            Ok(x) => Some(x),
            Err(_) => None, // the error is discarded
        }
    }
}

impl<T> Option<T> {
    fn ok_or<E>(self, err: E) -> Result<T, E> {
        match self {
            Some(v) => Ok(v),
            None => Err(err),
        }
    }

    fn ok_or_else<E, F>(self, err: F) -> Result<T, E>
    where
        F: FnOnce() -> E,
    {
        match self {
            Some(v) => Ok(v),
            None => Err(err()),
        }
    }
}
```

`#[must_use]` for `Result`

```
#[must_use = "this `Result` may be an `Err` variant, which should be handled"]
enum Result<T, E> {
    Ok(T),
    Err(E),
}

fn f() -> Result<(), ()> {
    Ok(())
}

f(); // warning: unused `Result` that must be used
let _ = f(); // explicitly ignored: no warning
```

Refactoring (0)

```
use std::fs::File;
use std::io::Read;

fn double_number_from_file(file_path: &str) -> i32 {
    let mut file = File::open(file_path).unwrap(); // error 1
    let mut contents = String::new();
    file.read_to_string(&mut contents).unwrap(); // error 2
    let n: i32 = contents.trim().parse().unwrap(); // error 3
    2 * n
}

let doubled = double_number_from_file("foobar");
println!("{}", doubled);
```

Refactoring (1): combinators

```
fn double_number_from_file(file_path: &str) -> Result<i32, String> {
    File::open(file_path)
        .map_err(|err| err.to_string())
        .and_then(|mut file| {
            let mut contents = String::new();
            file.read_to_string(&mut contents)
                .map_err(|err| err.to_string())
                .map(|_| contents)
        })
        .and_then(|contents| {
            contents
                .trim()
                .parse::<i32>()
                .map_err(|err| err.to_string())
        })
        .map(|n| 2 * n)
}

match double_number_from_file("foobar") {
    Ok(n) => println!("{}", n),
    Err(err) => println!("Error: {}", err),
}
```

Refactoring (2): early returns

```
fn double_number_from_file(file_path: &str) -> Result<i32, String> {
    let mut file = match File::open(file_path) {
        Ok(file) => file,
        Err(err) => return Err(err.to_string()),
    };
    let mut contents = String::new();
    if let Err(err) = file.read_to_string(&mut contents) {
        return Err(err.to_string());
    }
    let n: i32 = match contents.trim().parse() {
        Ok(n) => n,
        Err(err) => return Err(err.to_string()),
    };
    Ok(2 * n)
}
```

Propagate using `?`

```
// roughly: EXPR? ==
match EXPR {
    Ok(x) => x,
    Err(e) => return Err(From::from(e)), // `e.into()`
}

fn double_number_from_file(file_path: &str) -> Result<i32, String> {
    let mut file = File::open(file_path).map_err(|e| e.to_string())?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)
        .map_err(|e| e.to_string())?;
    let n = contents.trim().parse::<i32>().map_err(|e| e.to_string())?;
    Ok(2 * n)
}

fn main() -> Result<(), String> { // Termination trait
    double_number_from_file("foobar")?;
    Ok(())
}
```

Refactoring (3): define your own error type

```
#[derive(Debug)]
enum CliError {
    Io(std::io::Error),
    Parse(std::num::ParseIntError),
}

fn double_number_from_file(file_path: &str) -> Result<i32, CliError> {
    let mut file = File::open(file_path).map_err(CliError::Io)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents).map_err(CliError::Io)?;
    let n = contents.trim().parse::<i32>().map_err(CliError::Parse)?;
    Ok(2 * n)
}
```

Refactoring (4): type convertions

```
impl From<std::io::Error> for CliError {
    fn from(err: std::io::Error) -> CliError {
        CliError::Io(err)
    }
}

impl From<std::num::ParseIntError> for CliError {
    fn from(err: std::num::ParseIntError) -> CliError {
        CliError::Parse(err)
    }
}

fn double_number_from_file(file_path: &str) -> Result<i32, CliError> {
    let mut file = File::open(file_path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let n = contents.trim().parse::<i32>()?;
    Ok(2 * n)
}
```

`std::error::Error` trait

```
trait Error: Debug + Display {
    // The only method you may want to override
    fn source(&self) -> Option<&(dyn Error + 'static)> { None }

    // Deprecated: use `Display` / `source()` instead
    // fn description(&self) -> &str { ... }
    // fn cause(&self) -> Option<&dyn Error> { ... }

    // Unstable (nightly only)
    // fn provide<'a>(&'a self, request: &mut Request<'a>) { ... }
}
```

Manual `Display` and `Error` impl

```
use std::fmt;

impl fmt::Display for CliError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            CliError::Io(_) => write!(f, "failed to read data from file"),
            CliError::Parse(_) => write!(f, "failed to parse integer from string"),
        }
    }
}

impl std::error::Error for CliError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            CliError::Io(err) => Some(err),
            CliError::Parse(err) => Some(err),
        }
    }
}
```

Box<dyn Error>

```rust
use std::error::Error;

fn parse_port(s: &str) -> Result<u16, Box<dyn Error>> {
    let port: u16 = s.parse()?; // ParseIntError -> Box<dyn Error>
    if port == 0 {
        return Err("port must be non-zero".into()); // &str -> Box<dyn Error>
    }
    Ok(port)
}
```

- Любую ошибку (`impl Error`) и даже строку можно превратить в `Box<dyn Error>` через `From`, поэтому `?` работает «из коробки»
- Не нужны внешние зависимости, но тип ошибки стирается — вернуть конкретный вариант можно только через `downcast_ref`

Refactoring (5): `thiserror` crate

```
use thiserror::Error;
#[derive(Debug, Error)]
enum CliError {
    #[error("failed to read data from file: {0}")]
    Io(#[from] std::io::Error),
    #[error("failed to parse integer from string: {0}")]
    Parse(#[from] std::num::ParseIntError),
}

if let Err(err) = double_number_from_file("foobar") {
    println!("Program finished with error: {err}");
}
```

```
cargo run

Program finished with error: failed to read data from file: No such file or directory (os error 2)
```

Refactoring (5): `thiserror` crate

- Не просвечивается в публичном `API`: это просто derive-макрос, в сигнатурах ваших типов нет типов из `thiserror`.
- Эффект аналогичный ручной имплементации `From`, `Error`, `Display` (`Debug` по-прежнему выводится через `derive`).
- Подходит для использования в публичных библиотеках.

Refactoring (6): `anyhow` crate

```
use std::{fs::File, io::Read as _};
use anyhow::Result; // anyhow::Result<T> = Result<T, anyhow::Error>;
fn double_number_from_file(file_path: &str) -> Result<i32> {
    let mut file = File::open(file_path)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)?;
    let n = contents.trim().parse::<i32>()?;
    Ok(2 * n)
}
fn main() -> Result<()> {
    double_number_from_file("foobar")?;
    Ok(())
}
```

```
Error: No such file or directory (os error 2)
```

Refactoring (6): `anyhow` crate, add `Context`

```
use anyhow::{Context, Result};
fn double_number_from_file(file_path: &str) -> Result<i32> {
    let mut file = File::open(file_path)
        .with_context(|| format!("failed to open file `{file_path}`"))?;
    let mut contents = String::new();
    file.read_to_string(&mut contents)
        .with_context(|| format!("failed to read data from file `{file_path}`"))?;
    let contents = contents.trim();
    let n = contents
        .parse::<i32>()
        .with_context(|| format!("failed to parse `{contents}` as integer number"))?;
    Ok(2 * n)
}
```

```
cargo run
Error: failed to open file `foobar`

Caused by:
    No such file or directory (os error 2)
```

Refactoring (6): `anyhow` crate

- Основан на динамической диспетчеризации. Можно downcast'ить ошибки (`err.downcast_ref::<std::io::Error>()`).
- Работает с типами, которые реализуют `std::error::Error` (в том числе через `thiserror`).
- Сам `anyhow::Error` не реализует `std::error::Error`.
- Подходит для использования в конечных приложениях.

Which approach to choose?

	Когда	Плюсы	Минусы
`String` / `&str`	Прототип, пара строк	Ничего не нужно	Теряем тип и причину
Свой `enum` + `thiserror`	Библиотеки, код, где нужно реагировать на вид ошибки	Типизированные ошибки, `match`	Нужно описывать варианты
`Box<dyn Error>`	Скрипты, без зависимостей	Любая ошибка через `?`	Нет контекста, тип стёрт
`anyhow`	Приложения,	Контекст,	Тип стёрт, не для API

To `panic!` or Not to `panic!`

- Паника - выбор за двоих (вызываемый код решает за вызывающего)
- Примеры, прототипы, тесты - удобнее паниковать
- Иногда у компилятора меньше информации, чем у автора кода:

```
let home: std::net::IpAddr = "127.0.0.1" // valid IP address
    .parse()
    .expect("Hardcoded IP address should be valid");
```

- Если ошибка ожидаема на вызывающей стороне, стоит вернуть `Result`:
  - Ошибка парсинга JSON
  - HTTP-запрос завершился с кодом ответа, отличным от 200
  - В БД отсутствует пользователь с запрошенным `id`